

MACHINE LEARNING • SCIKIT-LEARN

Hiperparámetros en Machine Learning

Qué controlan, cuántos hay y cuáles conviene ajustar — una guía práctica por algoritmo.

[GridSearchCV](#) • [RandomizedSearchCV](#) • [Optimización Bayesiana](#)

No existe una lista única

Cada algoritmo de Machine Learning tiene **sus propios hiperparámetros**. No se ajustan igual y no todos pesan lo mismo: la clave es saber cuáles mueven la aguja.

Propios de cada modelo

Desde 2 en Regresión Lineal hasta más de 70 en CatBoost.

Pocos concentran el impacto

Entre 3 y 8 hiperparámetros explican la mayor parte del rendimiento.

Búsqueda automática

Grid Search, Random Search y Optimización Bayesiana exploran las combinaciones.

Modelos lineales, vecinos y árboles

Algoritmo	Hiperparámetros principales	¿Qué controlan?
Regresión Lineal	fit_intercept, positive	Intercepto, coeficientes positivos
Regresión Logística	C, penalty, solver, max_iter, class_weight	Regularización y entrenamiento
KNN	n_neighbors, weights, metric, p	Cantidad de vecinos, distancia
Árbol de Decisión	max_depth, min_samples_split, min_samples_leaf, criterion, max_features	Complejidad del árbol
Random Forest	n_estimators, max_depth, max_features, min_samples_leaf, bootstrap	Cantidad y tamaño de árboles
Extra Trees	n_estimators, max_depth, max_features	Similar a RF pero más aleatorio
SVM	C, kernel, gamma, degree	Margen y tipo de frontera
Naive Bayes	var_smoothing (Gaussian), alpha (Multinomial/Bernoulli)	Suavizado

Boosting y ensembles

Algoritmo	Hiperparámetros principales	¿Qué controlan?
Gradient Boosting	n_estimators, learning_rate, max_depth, subsample	Cantidad de árboles y aprendizaje
AdaBoost	n_estimators, learning_rate	Número de estimadores
HistGradientBoosting	learning_rate, max_depth, max_iter, l2_regularization	Boosting optimizado
XGBoost	eta, max_depth, subsample, colsample_bytree, gamma, lambda, alpha, n_estimators	Regularización y crecimiento
LightGBM	num_leaves, learning_rate, feature_fraction, bagging_fraction, max_depth	Boosting basado en hojas
CatBoost	depth, iterations, learning_rate, l2_leaf_reg	Boosting para variables categóricas

No supervisado y redes neuronales

Algoritmo	Hiperparámetros principales	¿Qué controlan?
K-Means	n_clusters, init, max_iter, n_init	Cantidad de clusters
DBSCAN	eps, min_samples, metric	Densidad del clustering
Agglomerative Clustering	n_clusters, linkage, metric	Agrupamiento jerárquico
PCA	n_components, svd_solver, whiten	Reducción de dimensionalidad
MLPClassifier	hidden_layer_sizes, activation, solver, learning_rate_init, alpha, batch_size, max_iter	Arquitectura y entrenamiento

Hiperparámetros que se repiten

Muchos algoritmos comparten los mismos hiperparámetros. Dominar estos te sirve para casi todos los modelos.

n_estimators

Cantidad de árboles

max_depth

Profundidad máxima

learning_rate

Velocidad de aprendizaje

max_iter

Máximo de iteraciones

alpha

Regularización L1/L2

lambda

Regularización L2 (XGBoost)

C

Inversa de la regularización

gamma

Influencia de cada punto

subsample

% de muestras utilizadas

max_features

Variables por división

min_samples_split

Mínimo para dividir un nodo

min_samples_leaf

Mínimo de muestras por hoja

criterion

Calidad de una división

kernel

Tipo de kernel (SVM)

n_neighbors

Cantidad de vecinos (KNN)

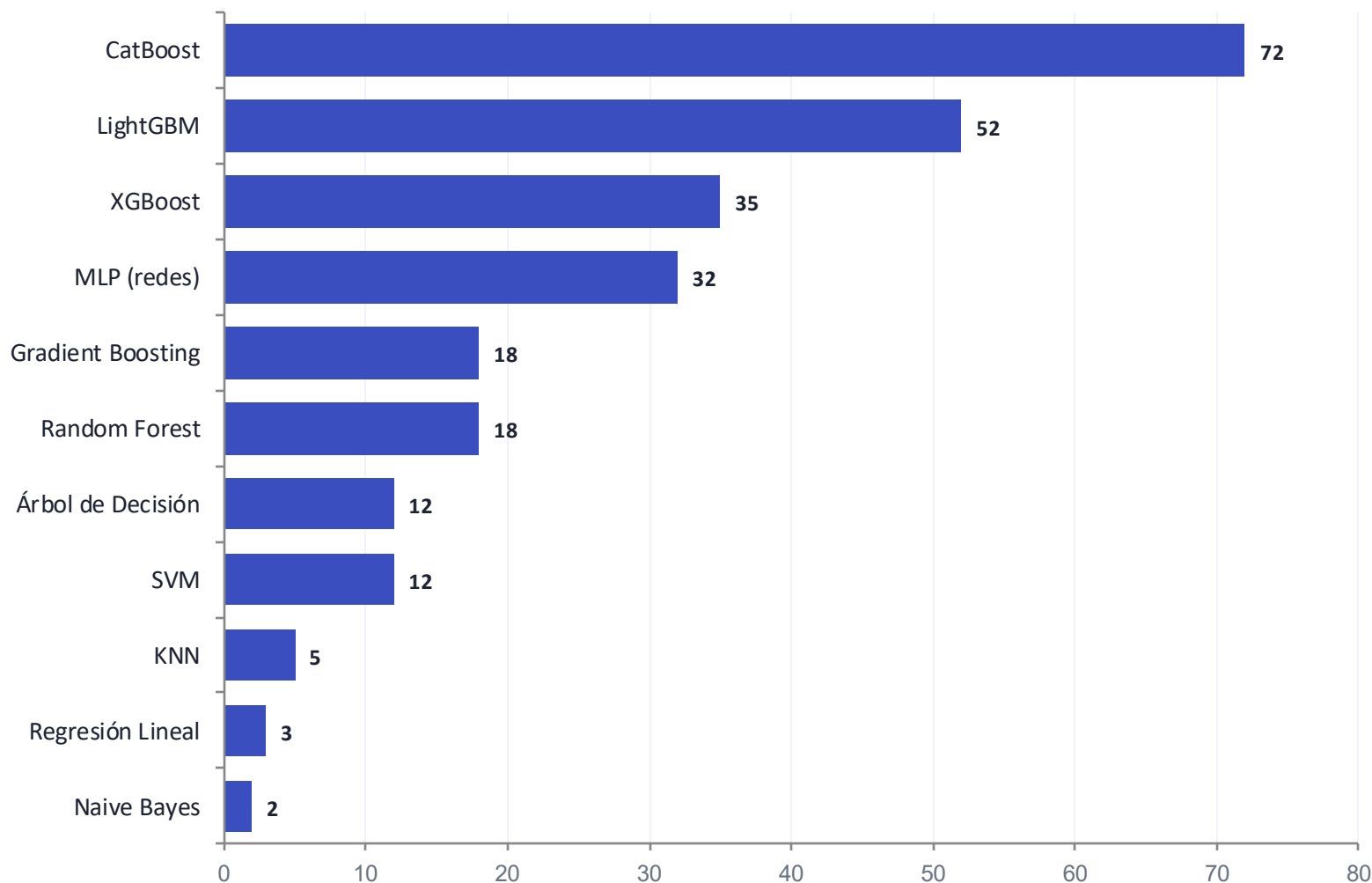
metric

Métrica de distancia

random_state

Semilla de reproducibilidad

¿Cuántos hiperparámetros tiene cada uno?



Valores aproximados, según implementación.

Más $\neq$ mejor

Aunque LightGBM y CatBoost superan los 50, no hace falta tocarlos todos.

Entre 3 y 8 hiperparámetros concentran la mayor parte del rendimiento.

¿Cuáles conviene ajustar?

Todos se pueden ajustar. La diferencia: unos se buscan con Grid/Random Search; otros se fijan una vez según el problema.

Algoritmo	● Conviene ajustar (tuning)	○ Configuración
Regresión Lineal	–	fit_intercept, positive
Regresión Logística	C, penalty, max_iter	solver, class_weight
KNN	n_neighbors, weights, metric, p	–
Árbol de Decisión	max_depth, min_samples_split, min_samples_leaf, max_features	criterion
Random Forest	n_estimators, max_depth, max_features, min_samples_leaf, min_samples_split	bootstrap
Extra Trees	n_estimators, max_depth, max_features	bootstrap
SVM	C, gamma, kernel, degree	–
Naive Bayes	alpha / var_smoothing	–
Gradient Boosting	n_estimators, learning_rate, max_depth, subsample	–
AdaBoost	n_estimators, learning_rate	–

Boosting avanzado y no supervisado

Algoritmo	● Conviene ajustar (tuning)	○ Configuración
HistGradientBoosting	learning_rate, max_depth, max_iter, l2_regularization	—
XGBoost	learning_rate, max_depth, subsample, colsample_bytree, gamma, lambda, alpha, n_estimators	—
LightGBM	num_leaves, learning_rate, feature_fraction, bagging_fraction, max_depth	—
CatBoost	depth, iterations, learning_rate, l2_leaf_reg	—
K-Means	n_clusters, init	max_iter, n_init
DBSCAN	eps, min_samples	metric
Agglomerative Clustering	n_clusters, linkage	metric
PCA	n_components	svd_solver, whiten
MLPClassifier	hidden_layer_sizes, activation, learning_rate_init, alpha, batch_size, max_iter	solver

En la práctica, casi siempre estos

<code>n_estimators</code>	Random Forest, XGBoost, LightGBM, AdaBoost, Gradient Boosting
<code>max_depth</code>	Árboles, Random Forest, XGBoost, LightGBM
<code>learning_rate</code>	Boosting, Redes Neuronales
<code>C</code>	Regresión Logística, SVM
<code>gamma</code>	SVM, XGBoost
<code>n_neighbors</code>	KNN
<code>n_clusters</code>	K-Means, Clustering jerárquico
<code>min_samples_leaf</code>	Árboles y Random Forest
<code>min_samples_split</code>	Árboles y Random Forest
<code>subsample</code>	Gradient Boosting, XGBoost, LightGBM
<code>hidden_layer_sizes</code>	Redes Neuronales (MLP)

≈ 80% del rendimiento, en pocos

Random Forest

`n_estimators · max_depth · max_features`

XGBoost

`learning_rate · n_estimators · max_depth · subsample
· colsample_bytree`

SVM

`C · gamma · kernel`

KNN

`n_neighbors · metric`

MLP (redes)

`hidden_layer_sizes · learning_rate_init · alpha`

Cómo buscarlos

Grid Search

Prueba todas las combinaciones de una grilla.

Random Search

Muestrea combinaciones al azar; eficiente.

Optimización Bayesiana

Aprende de intentos previos para elegir el siguiente.

SEGUNDA PARTE

Ficha por algoritmo

Qué es cada hiperparámetro, cuáles ajusta el programador en la práctica (● Ajustar / ○ Fijo), su rango típico y un ejemplo de código.

Regresión Lineal

Regresión · modelo lineal base

Hiperparámetro	¿Qué es?	Ajuste · rango
<code>fit_intercept</code>	Calcula el término independiente b_0	○ Fijo True (def.)
<code>positive</code>	Fuerza coeficientes ≥ 0	○ Fijo False (def.)

Ejemplo – scikit-learn / API

```
from sklearn.linear_model \
    import LinearRegression

model = LinearRegression(
    fit_intercept=True)
model.fit(X_train, y_train)

# ¿Regularizar?
# Ridge(alpha=1.0) o
# Lasso(alpha=0.1)
```

En la práctica: casi no se tunea; para regularizar se usa Ridge(alpha) o Lasso(alpha).

Regresión Logística

Clasificación · lineal con regularización

Hiperparámetro	¿Qué es?	Ajuste · rango
C	Inversa de la regularización (↓ = más fuerte)	● Ajustar 0.01-100 log
penalty	Tipo de penalización	● Ajustar l2 · l1
max_iter	Iteraciones hasta converger	● Ajustar 100-1000
solver	Algoritmo de optimización	○ Fijo lbfgs · saga
class_weight	Compensa clases desbalanceadas	○ Fijo None · balanced

Ejemplo - scikit-learn / API

```
param_grid = {  
    'C': [0.01, 0.1, 1, 10],  
    'penalty': ['l2'],  
    'max_iter': [200, 500],  
}  
GridSearchCV(  
    LogisticRegression(),  
    param_grid, cv=5)
```

K-Nearest Neighbors (KNN)

Clasificación / regresión · basado en distancia

Hiperparámetro	¿Qué es?	Ajuste · rango
n_neighbors	Cantidad de vecinos que votan (k)	● Ajustar 3-30
weights	Peso de los vecinos	● Ajustar uniform · distance
metric	Métrica de distancia	● Ajustar euclidean · manhattan
p	Potencia de Minkowski (1=L1, 2=L2)	● Ajustar 1 · 2

Ejemplo - scikit-learn / API

```
param_grid = {  
    'n_neighbors': range(3,31,2),  
    'weights':  
        ['uniform','distance'],  
    'p': [1, 2],  
}  
GridSearchCV(  
    KNeighborsClassifier(),  
    param_grid, cv=5)
```

En la práctica: escalá las variables antes; k impar evita empates en clasificación.

Árbol de Decisión

Clasificación / regresión · modelo de árbol

Hiperparámetro	¿Qué es?	Ajuste · rango
<code>max_depth</code>	Profundidad máxima; frena el sobreajuste	● Ajustar 3-20 · None
<code>min_samples_split</code>	Mín. muestras para dividir un nodo	● Ajustar 2-20
<code>min_samples_leaf</code>	Mín. muestras en una hoja	● Ajustar 1-10
<code>max_features</code>	Variables consideradas por división	● Ajustar sqrt · log2
<code>criterion</code>	Calidad de la división	○ Fijo gini · entropy

Ejemplo - scikit-learn / API

```
param_grid = {  
    'max_depth': [3,5,10,None],  
    'min_samples_leaf': [1,2,5],  
    'criterion':  
        ['gini','entropy'],  
}  
GridSearchCV(  
    DecisionTreeClassifier(),  
    param_grid, cv=5)
```


Random Forest

Clasificación / regresión · ensamble (bagging)

Hiperparámetro	¿Qué es?	Ajuste · rango
n_estimators	Cantidad de árboles del bosque	● Ajustar 100-1000
max_depth	Profundidad de cada árbol	● Ajustar 5-30 · None
max_features	Variables por división	● Ajustar sqrt · log2
min_samples_leaf	Mín. muestras por hoja	● Ajustar 1-5
min_samples_split	Mín. para dividir un nodo	● Ajustar 2-10
bootstrap	Muestreo con reemplazo	○ Fijo True (def.)

Ejemplo - scikit-learn / API

```
param_grid = {  
    'n_estimators': [200, 500],  
    'max_depth': [10, 20, None],  
    'max_features':  
        ['sqrt', 'log2'],  
}  
RandomizedSearchCV(  
    RandomForestClassifier(),  
    param_grid, cv=5)
```

Extra Trees

Clasificación / regresión · ensamble muy aleatorio

Hiperparámetro	¿Qué es?	Ajuste · rango
n_estimators	Cantidad de árboles	● Ajustar 100-1000
max_depth	Profundidad de cada árbol	● Ajustar 5-30 · None
max_features	Variables por división	● Ajustar sqrt · log2
min_samples_leaf	Mín. muestras por hoja	● Ajustar 1-5
bootstrap	Por defecto usa todo el set	○ Fijo False (def.)

Ejemplo - scikit-learn / API

```
param_grid = {  
    'n_estimators': [300, 600],  
    'max_depth': [None, 20],  
    'max_features': ['sqrt'],  
}  
GridSearchCV(  
    ExtraTreesClassifier(),  
    param_grid, cv=5)
```

En la práctica: divisiones al azar → más rápido y con menos varianza que Random Forest.

SVM (Support Vector Machine)

Clasificación / regresión · márgenes y kernels

Hiperparámetro	¿Qué es?	Ajuste · rango
C	Penalización por error vs. margen	● Ajustar 0.1-100 log
kernel	Forma de la frontera	● Ajustar rbf · linear · poly
gamma	Influencia de cada punto (rbf/poly)	● Ajustar scale · 0.001-1
degree	Grado del polinomio (solo poly)	● Ajustar 2-5

Ejemplo - scikit-learn / API

```
param_grid = {  
    'C': [0.1, 1, 10],  
    'kernel': ['rbf'],  
    'gamma':  
        ['scale', 0.01, 0.1],  
}  
GridSearchCV(  
    SVC(), param_grid, cv=5)
```

En la práctica: escalá siempre las variables; empezá con kernel rbf.

Naive Bayes

Clasificación · probabilístico, muy rápido

Hiperparámetro	¿Qué es?	Ajuste · rango
<code>var_smoothing</code>	Suavizado de varianza (GaussianNB)	● Ajustar 1e-9-1e-6 log
<code>alpha</code>	Suavizado de Laplace (Multinomial/Bernoulli)	● Ajustar 0.01-1

Ejemplo – scikit-learn / API

```
import numpy as np

# GaussianNB
{'var_smoothing':
 np.logspace(-9,-6,4)}

# MultinomialNB (texto)
{'alpha': [0.1, 0.5, 1.0]}
```

En la práctica: poquísimos hiperparámetros; excelente baseline para texto.

Gradient Boosting

Clasificación / regresión · boosting secuencial

Hiperparámetro	¿Qué es?	Ajuste · rango
<code>n_estimators</code>	Número de árboles secuenciales	● Ajustar 100-500
<code>learning_rate</code>	Contribución de cada árbol	● Ajustar 0.01-0.2
<code>max_depth</code>	Profundidad (árboles débiles)	● Ajustar 2-5
<code>subsample</code>	Fracción de muestras por árbol	● Ajustar 0.6-1.0

En la práctica: `learning_rate` y `n_estimators` se compensan: más árboles $\Rightarrow$ menor rate.

Ejemplo – scikit-learn / API

```
param_grid = {  
    'n_estimators': [100, 300],  
    'learning_rate': [0.05, 0.1],  
    'max_depth': [2, 3],  
    'subsample': [0.8, 1.0],  
}  
GridSearchCV(  
    GradientBoostingClassifier(),  
    param_grid, cv=5)
```

AdaBoost

Clasificación / regresión · boosting adaptativo

Hiperparámetro	¿Qué es?	Ajuste · rango
<code>n_estimators</code>	Número de estimadores débiles	● Ajustar 50-500
<code>learning_rate</code>	Peso de cada estimador	● Ajustar 0.01-1.0
<code>estimator</code>	Modelo base (árbol de profundidad 1)	○ Fijo stump (def.)

En la práctica: sensible al ruido; baja `learning_rate` si hay overfitting.

Ejemplo - scikit-learn / API

```
param_grid = {  
    'n_estimators': [50,100,200],  
    'learning_rate':  
        [0.1, 0.5, 1.0],  
}  
GridSearchCV(  
    AdaBoostClassifier(),  
    param_grid, cv=5)
```

HistGradientBoosting

Clasificación / regresión · boosting optimizado (sklearn)

Hiperparámetro	¿Qué es?	Ajuste · rango
<code>learning_rate</code>	Velocidad de aprendizaje	● Ajustar 0.01-0.3
<code>max_depth</code>	Profundidad (o sin límite)	● Ajustar None · 3-10
<code>max_iter</code>	Número de iteraciones (árboles)	● Ajustar 100-1000
<code>l2_regularization</code>	Regularización L2	● Ajustar 0-10

Ejemplo - scikit-learn / API

```
param_grid = {  
    'learning_rate': [0.05, 0.1],  
    'max_iter': [200, 500],  
    'l2_regularization': [0, 1],  
}  
GridSearchCV(  
    HistGradientBoosting  
        Classifier(), param_grid)
```

En la práctica: muy rápido y maneja NaN de forma nativa; alternativa a XGBoost/LightGBM.

XGBoost

Clasificación / regresión · boosting regularizado

Hiperparámetro	¿Qué es?	Ajuste · rango
<code>eta / learning_rate</code>	Velocidad de aprendizaje	● Ajustar 0.01-0.3
<code>max_depth</code>	Profundidad del árbol	● Ajustar 3-10
<code>subsample</code>	Fracción de filas por árbol	● Ajustar 0.6-1.0
<code>colsample_bytree</code>	Fracción de columnas por árbol	● Ajustar 0.6-1.0
<code>gamma</code>	Ganancia mínima para dividir	● Ajustar 0-5
<code>lambda</code>	Regularización L2	● Ajustar 0-10
<code>alpha</code>	Regularización L1	● Ajustar 0-10
<code>n_estimators</code>	Número de árboles	● Ajustar 100-1000

Ejemplo - scikit-learn / API

```
params = {  
    'eta': [0.05, 0.1],  
    'max_depth': [4, 6],  
    'subsample': [0.8],  
    'colsample_bytree': [0.8],  
    'lambda': [1, 5],  
}  
XGBClassifier(  
    n_estimators=500, **best)
```


LightGBM

Clasificación / regresión · boosting por hojas

Hiperparámetro	¿Qué es?	Ajuste · rango
<code>num_leaves</code>	Hojas por árbol (complejidad)	● Ajustar 15-255
<code>learning_rate</code>	Velocidad de aprendizaje	● Ajustar 0.01-0.3
<code>feature_fraction</code>	Fracción de columnas por árbol	● Ajustar 0.6-1.0
<code>bagging_fraction</code>	Fracción de filas por iteración	● Ajustar 0.6-1.0
<code>max_depth</code>	Profundidad (-1 = sin límite)	● Ajustar -1 · 3-12

Ejemplo - scikit-learn / API

```
params = {  
    'num_leaves': [31, 63],  
    'learning_rate': [0.05, 0.1],  
    'feature_fraction': [0.8],  
    'bagging_fraction': [0.8],  
}  
LGBMClassifier(  
    n_estimators=500, **best)
```

En la práctica: $\text{num_leaves} < 2^{\text{max_depth}}$ para no sobreajustar.

CatBoost

Clasificación / regresión · boosting para categóricas

Hiperparámetro	¿Qué es?	Ajuste · rango
depth	Profundidad de los árboles	● Ajustar 4-10
iterations	Número de árboles / iteraciones	● Ajustar 100-1000
learning_rate	Velocidad de aprendizaje	● Ajustar 0.01-0.3
l2_leaf_reg	Regularización L2 en las hojas	● Ajustar 1-10

Ejemplo - scikit-learn / API

```
params = {  
    'depth': [6, 8],  
    'learning_rate': [0.03, 0.1],  
    'l2_leaf_reg': [1, 3, 5],  
}  
CatBoostClassifier(  
    iterations=800,  
    cat_features=cats, **best)
```

En la práctica: maneja variables categóricas de forma nativa vía `cat_features`.

K-Means

Clustering · particional (no supervisado)

Hiperparámetro	¿Qué es?	Ajuste · rango
<code>n_clusters</code>	Número de grupos (k)	● Ajustar 2-10
<code>init</code>	Inicialización de centroides	○ Fijo k-means++
<code>n_init</code>	Reintentos con distinto init	○ Fijo 10 · auto
<code>max_iter</code>	Iteraciones por corrida	○ Fijo 300 (def.)

Ejemplo – scikit-learn / API

```
from sklearn.metrics import \
    silhouette_score

for k in range(2, 11):
    km = KMeans(n_clusters=k,
                init='k-means++')
    labels = km.fit_predict(X)
    score =
        silhouette_score(X, labels)
```

En la práctica: elegí k con el método del codo o silhouette_score.

DBSCAN

Clustering · por densidad (detecta ruido)

Hiperparámetro	¿Qué es?	Ajuste · rango
eps	Radio de vecindad	● Ajustar k-dist plot
min_samples	Mín. puntos para ser núcleo	● Ajustar 3-10
metric	Métrica de distancia	○ Fijo euclidean

Ejemplo – scikit-learn / API

```
from sklearn.cluster import DBSCAN
```

```
db = DBSCAN(  
    eps=0.5,  
    min_samples=5)  
labels = db.fit_predict(X)
```

```
# label = -1 ⇒ ruido
```

En la práctica: no fijás el nº de clusters; estimá eps con la curva de k-distancias.

Agglomerative Clustering

Clustering · jerárquico (aglomerativo)

Hiperparámetro	¿Qué es?	Ajuste · rango
<code>n_clusters</code>	Número de grupos a formar	● Ajustar 2-10
<code>linkage</code>	Criterio de fusión	● Ajustar ward · complete · average
<code>metric</code>	Distancia (ward $\Rightarrow$ euclidean)	○ Fijo euclidean

Ejemplo – scikit-learn / API

```
from sklearn.cluster import \
    AgglomerativeClustering

ac = AgglomerativeClustering(
    n_clusters=3,
    linkage='ward')
labels = ac.fit_predict(X)
```

En la práctica: explorá el dendrograma para decidir dónde cortar.

PCA

Reducción de dimensionalidad · no supervisado

Hiperparámetro	¿Qué es?	Ajuste · rango
n_components	Nº de componentes o varianza a conservar	● Ajustar 2-50 · 0.95
svd_solver	Algoritmo de descomposición SVD	○ Fijo auto · randomized
whiten	Normaliza los componentes	○ Fijo False (def.)

Ejemplo – scikit-learn / API

```
from sklearn.decomposition \
    import PCA

pca = PCA(n_components=0.95)
X_red = pca.fit_transform(X)

pca.explained_variance_ratio_
```

En la práctica: escalá los datos antes; n_components=0.95 conserva el 95% de varianza.

MLPClassifier (Red Neuronal)

Clasificación / regresión · perceptrón multicapa

Hiperparámetro	¿Qué es?	Ajuste · rango
hidden_layer_sizes	Arquitectura: capas y neuronas	● Ajustar (64,) · (128,64)
activation	Función de activación	● Ajustar relu · tanh
learning_rate_init	Tasa de aprendizaje inicial	● Ajustar 0.001-0.01
alpha	Regularización L2	● Ajustar 1e-5-1e-2
batch_size	Tamaño del lote	● Ajustar 32-256
max_iter	Épocas máximas	● Ajustar 200-1000
solver	Optimizador	○ Fijo adam (def.)

```
Ejemplo - scikit-learn / API

param_grid = {
    'hidden_layer_sizes':
        [(64,), (128,64)],
    'alpha': [1e-4, 1e-3],
    'learning_rate_init':
        [0.001, 0.01],
}
GridSearchCV(
    MLPClassifier(max_iter=500),
    param_grid, cv=3)
```

PARA LLEVAR

Lo esencial en tres ideas

1

Todos se pueden ajustar

La diferencia es cuáles vale la pena buscar (tuning) y cuáles se configuran una vez.

2

Pocos concentran el impacto

Entre 3 y 8 hiperparámetros explican ~80% del rendimiento en la mayoría de los modelos.

3

Empezá por el shortlist

Usá las listas por modelo y automatizá con Grid, Random o Optimización Bayesiana.